BKPilot

MVP — Front-end para Demonstração Comercial

Arquitetura, Proposta Técnica e Plano de Implementação

Documento gerado para o time comercial e técnico — guia de referência para a construção do produto mínimo viável de demonstração.

BugKillers — Setor de Inteligência Artificial

Abril 2026 | Documento Confidencial

---

1. Resumo Executivo

O BKPilot é uma plataforma de QA automatizado baseada em Inteligência Artificial. Hoje opera inteiramente via linha de comando — ideal para times técnicos, mas inviável para demonstrações comerciais a clientes não técnicos.

Este documento descreve a proposta de criação de um MVP (Produto Mínimo Viável) com interface web que esconde toda a complexidade técnica, expondo apenas três campos e um botão para o usuário final.

---

2. O Problema Atual

Como funciona hoje

O operador abre o terminal e executa os comandos na seguinte ordem:

```
/plano-automacao    -> avalia viabilidade e estima investimento
/explorar           -> mapeia o sistema alvo
/gerar-cenarios     -> cria os cenarios de teste
/testar-modulo      -> executa os testes
/reportar-bug       -> documenta os bugs encontrados
/gerar-relatorio    -> gera o relatorio PDF final
```

Cada skill é um passo do trabalho de QA. O operador precisa conhecer os comandos, entender o output técnico e tomar decisões durante o processo. Isso torna a demonstração comercial impraticável — o cliente veria apenas código rolando na tela.

---

3. A Solução Proposta

3.1 Skill `/pipeline-completo`

Uma nova skill que encadeia todos os passos em sequência automaticamente. O front-end envia apenas três dados:

- URL do site a ser testado
- Usuário de login
- Senha de acesso

A senha é fornecida pelo cliente na hora da visita — cenário real de venda em que o sistema do cliente nunca foi acessado antes. O comercial digita no front-end.

E ela executa sozinha: plano de automação → explorar → gerar cenários → testar → reportar bugs → gerar PDF. É o equivalente a um botão "Fazer Tudo" que substitui os comandos manuais.

Segurança da senha: a senha NUNCA é passada como argumento de linha de comando (apareceria em `ps aux`). A API a injeta como variável de ambiente (`QA_PASSWORD`) no processo filho do Claude Code — igual o motor já faz com `.env`. A senha nunca é logada, nunca vai para o `progresso.json` e expira da memória quando o processo termina.

> ***Nota sobre estrutura multi-tenant:*** *O MVP utiliza um cliente fixo `demo_mvp` dentro da estrutura `clients/` do projeto BKPilot. A API cria a pasta `clients/demo_mvp/` (se não existir) e grava `progresso.json` em `clients/demo_mvp/resultado/<timestamp>/`, respeitando a estrutura multi-tenant definida no `AGENTS.md`. A senha recebida via POST é injetada como variável de ambiente; o arquivo `clients/demo_mvp/.env` é preenchido temporariamente pela API e removido após a execução.*

3.2 Front-end Web (a casca bonita)

Uma tela web simples acessada pelo navegador. O cliente vê apenas um formulário limpo, uma barra de progresso e um botão de download do relatório. Toda a complexidade do Claude Code + skills fica completamente oculta.

Estados da interface:

- **Não autenticado:** tela de login com autenticação Microsoft 365 (e-mail `@bugkillers.com.br` via Microsoft Entra ID). Sem autenticação, o formulário de demo não é exibido.
- **Inicial:** formulário com 3 campos + botão "Testar"
- **Ocupado:** mensagem "Sistema em uso, aguarde..." (se outro vendedor estiver usando)
- **Executando:** barra de progresso + nome da etapa atual
- **Falha:** mensagem de erro + botão "Tentar novamente" (1 retry)
- **Concluído:** botão "Baixar Relatório PDF"

> ***Autenticação obrigatória:*** *O acesso à tela de demo requer login com conta Microsoft 365 da BugKillers. Isso impede uso indevido por terceiros e protege o consumo de tokens da Anthropic.*

---

4. Arquitetura na VPS

Três serviços rodando no servidor

```
+-------------------------------------------------------------+
|                          SUA VPS                            |
|                                                             |
|  +------------+    +------------+    +-----------------+   |
|  |  CAIXA 1   |    |  CAIXA 2   |    |    CAIXA 3      |   |
|  |  Front-end | -> |    API     | -> |     Motor       |   |
|  |  (a tela)  | <- | (o garcom) | <- |  Claude Code    |   |
|  |  Next.js   |    |  Node.js   |    |  + Playwright   |   |
|  |  :3000     |    |   :8080    |    |  (background)   |   |
|  +------------+    +------------+    +-----------------+   |
+-------------------------------------------------------------+
          ^
          | internet
     +----+----+
     | Vendedor|  (abre no browser)
     +---------+
```

Caixa 1 — Front-end (a tela)

- Tecnologia: Next.js ou React
- Exibe: formulário (URL, login, senha), barra de progresso, botão "Baixar PDF"
- Acesso: https://demo.bugkillers.com.br
- Autenticação: Microsoft 365 (Microsoft Entra ID) — apenas contas `@bugkillers.com.br`

Caixa 2 — API (o garçom)

- Tecnologia: Node.js com Express ou Fastify (~250 linhas de código)
- Recebe pedidos do front-end e repassa ao motor
- Mantém lock em **arquivo no disco** (`/tmp/bkpilot.lock`): apenas uma execução por vez, sobrevive a reinício da API
- **Validação de autenticação:** a API valida o token JWT da Microsoft 365 recebido no header `Authorization: Bearer <token>` em todos os endpoints. Sem token válido, retorna `401 Unauthorized`

Controle de concorrência: se chegar um segundo `POST /executar` enquanto outra demo está rodando, a API responde `{ status: "ocupado", estimativa: "calculando..." }` . O front-end mostra "Sistema em uso, aguarde...". Sem fila — recusar com estimativa resolve para MVP. A estimativa é baseada na duração da última execução concluída (mínimo 3 minutos).

Três endpoints expostos:

> *O `id` da execução é um **UUID v4** (`crypto.randomUUID()`), usado para nomear a pasta `clients/demo_mvp/resultado/<id>/` e rastrear o `progresso.json` .*

Endpoint	Método	Descrição
`/executar`	POST	recebe `{ url, login, senha }` + `Authorization: Bearer <token>` (JWT Microsoft 365), inicia teste, retorna `{ id }` (UUID v4). Se ocupado, retorna `{ status: "ocupado" }`
`/status/:id`	GET	consultado a cada 2s, lê `progresso.json` da pasta de execução. Retorna `{ etapa, progresso, erro }`
`/relatorio/:id`	GET	ao terminar, retorna o PDF para download

Caixa 3 — Motor (Claude Code rodando escondido)

- A API cria um processo filho do Claude Code em modo headless (sem interface)
- Credenciais (`QA_PASSWORD` , `QA_URL` , `QA_LOGIN`) são passadas como **variáveis de ambiente** do processo filho — nunca como argumentos de CLI
- Roda a skill `/pipeline-completo` em background
- Escreve resultados em `clients/<id>/resultado/<timestamp>/` — exatamente como hoje
- A cada etapa concluída, a skill escreve `progresso.json` na pasta de execução
- O processo filho é executado dentro do diretório raiz do projeto BKPilot (`/opt/bkpilot/` na VPS), garantindo acesso ao `CLAUDE.md` , `AGENTS.md` , `.claude/settings.json` , skills em `.claude/commands/` e ao diretório `clients/`

Mecanismo de progresso — `progresso.json` :

A skill `/pipeline-completo` grava este arquivo ao final de cada etapa, eliminando a necessidade de parsear logs:

```json
{
  "etapa": "gerando cenarios",
  "passo": 3,
  "totalPassos": 6,
  "progresso": 50,
  "inicio": "2026-04-25T14:30:00Z",
  "erro": null
}
```

A API simplesmente lê este JSON no endpoint `/status/:id`. Sem parsing frágil de logs.

---

5. Fluxo de uma Demonstração

Passo a passo do que acontece quando o vendedor usa o sistema:

1. **Vendedor abre o browser**, preenche os 3 campos e clica "Testar"
2. **Front-end envia:** `POST /executar { url, login, senha }` (HTTPS — tráfego cifrado)
3. **API responde:** `{ id: "abc123" }` (ou `{ status: "ocupado" }` se já houver demo rodando)
4. **API dispara Claude Code** em background com credenciais como variáveis de ambiente
5. **Front-end consulta a cada 2s:** `GET /status/abc123`
   - `{ etapa: "Avaliando viabilidade de automacao...", progresso: 10 }`
   - `{ etapa: "Explorando o sistema...", progresso: 25 }`
   - `{ etapa: "Gerando cenarios de teste...", progresso: 45 }`
   - `{ etapa: "Executando testes...", progresso: 70 }`
   - `{ etapa: "Gerando relatorio...", progresso: 90 }`
6. **Em caso de falha:** `{ etapa: "falha", erro: "Claude Code interrompeu durante exploracao", recuperavel: true }`
   - Front-end mostra mensagem de erro + botão "Tentar novamente" (1 retry)
   - **Timeout:** o pipeline inteiro tem limite de 30 minutos. Além disso, se `progresso.json` não alterar o campo `passo` em 10 minutos, a API marca falha por inatividade de etapa
7. **Quando termina com sucesso:** `{ etapa: "Concluido", progresso: 100 }`
8. **Aparece botão** "Baixar Relatorio PDF"
9. `GET /relatorio/abc123` → download do PDF gerado

---

6. Flexibilidade de Motor

A API foi pensada para ser "agnóstica" — não sabe qual motor está rodando por baixo. Hoje é Claude Code. Amanhã pode ser outro. O front-end nunca muda. As pastas `dist/claude/`, `dist/codex/` e `dist/opencode/` já existem no projeto, confirmando que as skills foram preparadas para múltiplos motores.

CLI / Motor	Como funciona	Vantagem	Desvantagem
Claude Code (atual)	Roda Opus/Sonnet Anthropic	Skills prontas, melhor qualidade	Custo por token
Codex CLI (OpenAI)	Roda GPT-5	Alternativa de custo	Skills precisam adaptação

OpenCode	Open-source, qualquer modelo	Flexível, troca sem mudar código	Menos polido
Agent SDK direto	Node.js + API Anthropic	Controle total, sem CLI	Reimplementa orquestração
Modelo local (Ollama)	Roda na própria VPS	Custo zero por uso	Qualidade inferior

Analogia: é como um carro elétrico. O motorista dirige o carro (front-end). O motor pode ser bateria da Tesla, BYD ou CATL — o painel é o mesmo. Você troca o motor sem refazer a tela.

---

7. Infraestrutura na VPS

Requisitos mínimos de hardware

- **RAM:** mínimo 8 GB (Playwright + Chromium headless podem consumir 2–3 GB em sistemas-alvo complexos; 4 GB é insuficiente para SPAs grandes)
- **CPU:** 2 vCPUs
- **Disco:** 20 GB (logs, screenshots, PDFs gerados)
- **Sistema operacional:** Ubuntu 22.04 LTS recomendado

Stack de infraestrutura

- **PM2** — mantém front-end e API vivos, reinicia automaticamente em caso de falha
- **Nginx** — HTTPS na frente, roteia `/` → front-end e `/api` → API
- **Certbot** — certificado SSL gratuito via Let's Encrypt
- **Arquivo `.env`** — chaves da API Anthropic e credenciais QA, nunca no repositório

Política de limpeza de disco

- Ao iniciar nova execução, a API remove pastas antigas do cliente, mantendo apenas as **2 execuções mais recentes**
- Verificação pré-execução: se disco livre < 2 GB, API recusa e retorna `{ status: "erro", motivo: "disco_cheio" }`
- Vídeos `.webm` (gerados via `recordVideo` — **camelCase obrigatório**; `record_video` em snake_case é silenciosamente ignorado pelo Playwright em Node.js) são removidos após conversão para `.mp4` e download do PDF

Configuração Nginx (resumo)

```
server {
    server_name demo.bugkillers.com.br;
    location /api/ {
        proxy_pass http://localhost:8080;
    }
    location / {
        proxy_pass http://localhost:3000;
    }
}
```

---

8. Plano de Implementação — MVP em 2 Semanas

Semana 1

- **Dias 1–2:** criar skill `/pipeline-completo` encadeando as 6 skills existentes, com gravação de `progresso.json` a cada etapa
- **Dias 3–4:** criar API Node.js com os 3 endpoints, lock em arquivo, timeout por etapa (10 min) e global (30 min), sanitização de logs
- **Dia 5:** testes de integração API ↔ Claude Code na VPS, incluindo cenários de falha

Semana 2

- **Dias 6–7:** front-end (formulário, barra de progresso, estados de erro/ocupado, download PDF)
- **Dia 8:** autenticação Microsoft 365 — App Registration no Azure AD / Microsoft Entra, fluxo OAuth2 (Authorization Code com PKCE), callback URL, troca de token, validação de JWT na API, tela de login
- **Dia 9:** configurar Nginx + PM2 + HTTPS + política de limpeza de disco na VPS
- **Dia 10:** testes end-to-end + ajustes de UX para demo + teste de retry e concorrência

Estimativa de esforço por componente

Componente	Esforço
Skill `/pipeline-completo` (com `progresso.json`)	1–2 dias
API Node.js (com lock, timeout, sanitização)	2–3 dias (~250 linhas de código)
Front-end (com estados de erro/ocupado)	3–4 dias (~400 linhas de código)
Autenticação Microsoft 365 (Entra ID + OAuth + JWT)	1 dia
Configuração de VPS (com política de limpeza)	1 dia
Total	**~2 semanas com 1 desenvolvedor**

---

9. As 19 Skills do BKPilot

O framework BKPilot opera com **19 skills principais** cobrindo todo o pipeline de QA:

Pipeline principal

Skill	Função
`/plano-automacao`	Consultoria de viabilidade técnica antes de investir em automação
`/explorar`	Mapeamento completo do sistema alvo
`/gerar-cenarios`	Geração de cenários estruturados (suporta Gherkin)
`/testar-modulo`	Execução por módulo com roteiro + exploração livre
`/executar-planilha`	Execução em lote com retry e circuit breaker
`/reportar-bug`	Consolidação de falhas em bug cards
`/gerar-relatorio`	Relatório final em PDF ou DOCX

`/relatorio-parcial`	Acompanhamento semanal em projetos longos

Skills de automação do cliente

Skill	Função
`/gerar-automacao-cliente`	Gera pacote de código no stack do cliente
`/auditar-automacao-cliente`	Auditoria independente com remediação obrigatória

Skills avulsas

Skill	Função
`/executar-fluxo`	Fluxo E2E ad-hoc (suporta data-driven)
`/testar-forms`	Formulário específico (13 grupos de teste)
`/regressao`	Reteste após correção (visual-diff + detecção de flaky)
`/acessibilidade`	Auditoria WCAG 2.1 (níveis A/AA/AAA)
`/performance`	Core Web Vitals (LCP, FCP, CLS, TTFB, INP)
`/api-check`	Testes de API (segurança, auth, payloads, rate limiting)
`/usabilidade`	Avaliação heurística de UX (10 heurísticas de Nielsen)
`/testar-ia`	Testes em fluxos de IA (jailbreak, hallucination, guardrails)
`/push-bugs`	Publicação de bugs em Jira/GitHub Issues

---

10. Analogias para o Time Comercial

Analogia do Uber

Por baixo do Uber existe algoritmo de roteamento, cálculo de preço dinâmico, GPS em tempo real e sistema de pagamento — tudo extremamente complexo. Mas o usuário só vê: "para onde vai?" e "pedir carro". O BKPilot MVP funciona da mesma forma: toda a complexidade fica escondida atrás de 3 campos e 1 botão.

Analogia do Carro Elétrico

O motorista dirige o carro (front-end). O motor pode ser bateria da Tesla, da BYD ou da CATL — tanto faz para quem dirige. O painel é exatamente o mesmo. Na nossa solução, a API é o "painel": você troca o motor (Claude Code → outro modelo) sem precisar reescrever a tela.

---

11. Salvaguardas e Tratamento de Bordas

O MVP inclui desde o início os seguintes mecanismos de robustez:

11.1 Segurança da senha do cliente

Camada	Regra

Front-end → API	HTTPS obrigatório. Campo `type="password"`. Senha trafega cifrada no body do POST
API	NUNCA loga a senha. Mantém em memória. Não salva em disco nem no `progresso.json`
API → Motor	Passa como **variável de ambiente** (`QA_PASSWORD`) do processo filho. NUNCA como `--senha abc123` na CLI
Motor (Claude Code)	Lê `process.env.QA_PASSWORD` — idêntico ao funcionamento atual com `.env`
Pós-execução	Processo filho morre → variável de ambiente some da memória. Sem rastros
API → Front-end (stdout/stderr)	A API intercepta todo output do motor e aplica **filtro regex** para remover `QA_PASSWORD`, `QA_LOGIN`, tokens de API e paths internos (`clients/<id>/`) antes de enviar ao front-end ou gravar em log

11.2 Concorrência — lock simples

A API mantém lock via **arquivo no disco** (`/tmp/bkpilot.lock`), contendo o PID do processo atual. Isso sobrevive a reinícios da API e evita locks órfãos:

```
const lockFile = '/tmp/bkpilot.lock';

function isLocked() {
  if (!fs.existsSync(lockFile)) return false;
  const pid = parseInt(fs.readFileSync(lockFile, 'utf8'));
  try {
    process.kill(pid, 0); // verifica se o processo ainda existe
    return true;
  } catch {
    fs.unlinkSync(lockFile); // processo morto, limpa lock
    return false;
  }
}

POST /executar
  if (isLocked()) return { status: "ocupado", estimativa: "calculando..." };
  fs.writeFileSync(lockFile, String(process.pid));
  // ... executa pipeline
  // ao terminar: fs.unlinkSync(lockFile);
```

O front-end exibe "Sistema em uso, aguarde..." com a estimativa baseada na última execução concluída. Fila completa é desnecessária para MVP.

> ***Nota para v2:*** *o lock em arquivo local (`/tmp/bkpilot.lock`) funciona para uma única instância da API. Se no futuro o PM2 for configurado em modo cluster (múltiplos processos Node.js), será necessário migrar para um lock distribuído (ex: Redis, PostgreSQL advisory lock ou mutex em NFS).*

11.3 Tratamento de falhas no pipeline

Três mecanismos independentes:

Mecanismo	Gatilho	Resposta
Timeout por etapa	`progresso.json` sem alteração de `passo` há 10 minutos	API marca `etapa: "falha"`, front-end mostra erro + botão "Tentar novamente"
Timeout global	Pipeline inteiro excede 30 minutos	API força encerramento do processo filho e marca falha
Exit code	Processo filho termina com exit ≠ 0	API registra falha no `progresso.json` com `erro: "<mensagem>"`
Estado de erro	Skill escreve `"erro": "..."` no `progresso.json` antes de morrer	API repassa ao front-end; se `recuperavel: true`, botão de retry habilitado

Retry limitado a **1 tentativa**. Se falhar de novo, mensagem "Entre em contato com o suporte técnico".

> ***Limitação conhecida do MVP:*** *apenas 1 retry por execução. Em cenários de instabilidade temporária do sistema-alvo (ex: indisponibilidade de rede do cliente), a demo pode falhar sem recuperação automática. Para demos críticas, o vendedor deve reiniciar manualmente.*

11.4 Tracking de progresso — `progresso.json`

A skill `/pipeline-completo` grava a cada etapa:

```
{
  "etapa": "gerando cenarios",
  "passo": 3,
  "totalPassos": 6,
  "progresso": 50,
  "inicio": "2026-04-25T14:30:00Z",
  "erro": null
}
```

A API lê este arquivo no `GET /status/:id` — sem depender de parsing frágil de logs.

11.5 Limpeza de disco

- Retenção: apenas as **2 execuções mais recentes** por cliente — pastas antigas removidas ao iniciar nova demo
- Pré-execução: se disco livre < 2 GB, recusa com alerta no front-end
- Vídeos `.webm` (gerados via `recordVideo` — **camelCase obrigatório**) removidos após conversão para `.mp4` e download do PDF
- Sempre que possível, a skill chama `page.close()` e `context.close()` para liberar recursos do Chromium

---

12. Próximos Passos

- [ ] Validar escopo do MVP com o time comercial
- [ ] Definir URL de demo (ex: demo.bugkillers.com.br)
- [ ] Iniciar desenvolvimento da skill `/pipeline-completo` (com gravação de `progresso.json`)
- [ ] Desenvolver API Node.js (com lock de concorrência, timeout, leitura de `progresso.json`)

- [ ] Implementar middleware de sanitização na API: regex para remover `QA_PASSWORD`, `QA_LOGIN`, tokens de API e paths internos (`clients/<id>/`) antes de enviar ao front-end ou gravar em log
- [ ] Desenvolver front-end web (com estados: carregando, ocupado, falha, concluído)
- [ ] Deploy na VPS + configuração de HTTPS + política de limpeza de disco
- [ ] Sessão de testes com o time de vendas antes da primeira demonstração
- [ ] Testar cenário de falha: matar processo Claude Code e verificar retry no front-end
- [ ] Testar cenário de concorrência: duas abas simultâneas e verificar lock

---